

钱进 - Resume

钱进

2026-06-15

钱进

客户端开发工程师 · 10 年经验 · 现居北京

qianjinchaoyue1111@163.com · qianjin1111.github.io/EddieResume

个人简介

10 年移动客户端研发经验，长期聚焦 **Android 原生 + HarmonyOS + 跨端 (KMP/Compose)**，能独立完成从需求梳理、方案设计、核心编码、联调到发布及线上质量治理的闭环。

- 2021.04 起就职于 **懂车帝**（至今在职）
- 既做过大跨度系统版本升级、多设备适配与业务需求迭代，也主导过冷启动 / 卡顿 / 内存 / 包体积等性能专项
- 擅长用 Profiler / 抓栈 / Hook 等手段定位疑难问题并形成可复用方法论
- 持续沉淀鸿蒙工程基础能力、一多与体验提升、性能优化、工程体验流程等实践文档

专业技能

平台与语言

精通 **Java / Kotlin**，熟悉 ArkTS；具备 Kotlin Multiplatform / Compose 跨端实战经验

Android 客户端与架构

熟悉 Activity/Fragment、View 体系与复杂列表构建，掌握 **MVVM、组件化/模块化分层**，具备折叠屏 / PAD / 横竖屏等多形态 UI 适配经验

网络通信

熟练基于 HTTP/HTTPS、WebSocket 进行接口通信与长连接管理，了解 TCP/UDP 基础与常见网络问题定位

数据存储与本地能力

熟悉 SharedPreferences / 文件 / SQLite 等本地存储，掌握 Keva、AutoDatabase 等 K-V / ORM 方案

性能与稳定性

- Android/HarmonyOS 冷启动、ANR/卡顿/掉帧、内存/OOM、包体积与功耗优化
- 熟悉 Native Crash 分析（信号/寄存器/Tombstone/backtrace）、ANR 归因、Java/Native OOM 分类与治理
- 理解 **Hook 技术**（PLT/GOT hook、Inline hook、Java/ART Hook），结合 bytehook/shadowhook 做监控与问题定位
- 熟悉 Slardar/NPTH、btrace、Sliver、Raphael、Tailor 等监控与 Trace 工具

工程化与发布

掌握多分支协作与 MR 流程，熟悉灰度发布与稳定发布节奏；主导过 IDE/SDK 大版本升级

AI 研发提效

具备 **AI Agent / Spec-Coding 规格化编码** 工程化落地经验，自研 Android→鸿蒙 AI 辅助转换（A2H）方案；熟悉映射知识库（RAG）、独立验证与同口径度量等大模型应用工程化手段

项目经历

懂车帝 HarmonyOS 版从 0 到 1 建设与追平主端

背景 (Situation)：懂车帝主端有 200w+ Java/Kotlin 代码、400+ 原生页面、270+ 版本迭代沉淀，鸿蒙生态早期基础库匮乏、一年内经历 15 个系统大版本，需在有限人力下追赶主端功能并保障体验。

职责 (Task)：作为鸿蒙端核心开发与项目推进者，负责需求追赶的整体节奏把控、跨团队协作与人力协调，并对版本质量与上线节奏负责。

举措 (Action)：

- **项目管理：**建立「需求分级（核心场景优先）+ 季度目标动态调整 + 复盘优化」机制，维护版本需求池（优先级 / 排期 / 测试工作量 / 风险），以「跟随主端约 2 版本、2-3 周滚动一版」的稳定节奏推进追赶。
- **协助与人力协调：**推动「固定 + 灵活人力协同、高阶同学双线并行、培养后备力量」的人员配置，把线上问题按 ≥ 0.5 人日门槛统一转为需求并由「需求周」统筹分配。
- **跨团队沟通：**牵头鸿蒙小周会与周报机制，串联 KMP / QA / UED / Server 多团队；作为对接窗口与华为一多负责人对齐评分目标、闭环 85+ 条华为 IR 问题，与华为共建汽车领域性能 S 标。
- **质量卡口：**落地「线上反馈 → 归因 → 转需求排版 → 测试周自动出包回归」闭环与防逃逸机制，主导 API17 → API20 大版本升级的独立灰度节奏。

结果 (Result)：10 个月完成鸿蒙版从 0 到 1，成为汽车垂类首个上架 App，实现 85%+ 版本还原度；一多多设备体验评分双折叠 4.79、三折叠 4.54 领先竞品；项目荣获华为「鸿蒙先锋-生态贡献奖」。

Android → 鸿蒙 AI 辅助转换 (A2H) 研发提效

背景 (Situation)：鸿蒙端追平主端需将大量 Android 页面（命令式 View + Activity/Fragment）改写为鸿蒙（声明式 ArkTS/ArkUI + 全新状态管理范式），两端范式差异大，纯人工逐页迁移是最大人力黑洞；而直接用通用大模型「直翻」存在编译不过、状态/语义漂移、结果不可验收三大硬伤，返工成本常高于重写。

职责 (Task)：借鉴 Spec-Coding（规格驱动编码）流水线思路，自研并落地一套面向 Android→鸿蒙（ArkTS/ArkUI）的 AI 辅助转换方案 (A2H)，在保证 UI 还原度与可编译的前提下显著提升转换效率，并把转换经验沉淀为团队资产。

举措 (Action)：

- **规格化状态机防幻觉：**将转换拆为「源码事实抽取（前置分析 + 独立审计）→ 规格（需求/设计/任务）→ 实现 → 独立验证」链路，先文档后代码、审批闸门、白名单产物，杜绝大模型越权乱改。
- **Android→鸿蒙映射知识库（核心 IP）：**将控件、API、生命周期、状态管理（MVVM LiveData/StateFlow → ArkUI @State/@Link/@Prop 单向数据流）等跨范式映射显式化为可检索（RAG）+ 可校验的知识库，并随人工审校持续回填增值。
- **复用鸿蒙 + KMP 双仓收敛翻译面：**业务逻辑经 KMP 共享后，AI 只需翻译 UI 与平台适配层，大幅降低幻觉与编译失败率（架构决定 AI 上限）。
- **实现与验证分离 + 自动回环：**规格契合度审计 → 编译校验 → UI 还原度/设备验收 串行独立兜底，失败自动回实现修复并复验；统一「单页转换耗时 / 一次编译通过率 / UI 还原度 / 人工修正率」同口径度量。

结果 (Result)：把单页/单组件转换从「资深人肉」升级为「AI 产线 + 人工审校」，单页转换效率提升约 N 倍（待按实测填写），一次编译通过率与 UI 还原度显著提升、返工大幅下降；沉淀可复用、可增值的 Android→鸿蒙映射知识库与转换 SOP，降低新人门槛，加速鸿蒙端追平主端。

客户端性能与稳定性优化专项 (Android → 鸿蒙)

背景 (Situation)：性能与稳定性优化贯穿我在懂车帝的不同阶段，且理论与方法论一脉相承，区别只在于平台与工具随阶段切换。早期在 Android 主端（200w+ 代码、400+ 原生页面），围绕 Native Crash、ANR、OOM、启动慢、卡顿丢帧等高频问题打磨问题定位与优化方法；后期鸿蒙端从 0 到 1 追平主端，生态早期工具链不成熟、缺乏统一口径，需要把已验证的方法论迁移到新平台并系统化落地。

职责 (Task)：

- **Android 阶段：**作为核心开发参与并负责过若干性能与稳定性 Topic（Native Crash / ANR / OOM / 卡顿定位等），沉淀通用排查方法与工具使用经验。
- **鸿蒙阶段：**作为鸿蒙端性能与稳定性专项 Owner，整体负责专项目标制定、口径统一、流程与机制建设，并对核心性能 / 稳定性指标负责。

举措 (Action)：

- **方法论一致、工具随平台切换：**两阶段共用同一套「线上发现 → 线下归因 → 优化 → 同口径复测」闭环方法论。Android 侧依托 Slardar 聚类发现线上问题，结合 Tombstone、ANR traces、Perfetto、Memory Profiler、btrace / Sliver 等证据归因，沉淀排查手册与复盘模板。
- **鸿蒙专项体系化 (Owner 主导)：**线上以 Slardar 为主发现与监控问题，线下以 DevEco Profiler 与 SmartPerf 为

主要工具治理冷启动、卡顿丢帧与内存；通过 **Launch** 拆解启动阶段耗时并将启动任务按「首屏必需 / 非必需」分级；通过 **Frame / Trace** 定位复杂页面掉帧根因。

- **机制与度量**：建立**每日性能优化日报**与**同口径复测机制**，保证优化前后可对比、问题可追踪、收益可量化，避免劣化反复。
- **跨平台经验迁移**：将 Android 成熟的抓栈 / 监控 / 归因思路迁移到鸿蒙（如高性能抓栈、混合栈 Trace 方案），降低新平台从 0 建设成本。

结果 (Result)：以一致方法论支撑 Android 与鸿蒙双平台的性能与稳定性治理；在鸿蒙端作为专项 Owner 建立起统一口径的问题定位与优化体系，性能优化日报与同口径复测机制常态化运行，核心高频问题实现「可发现、可归因、可复测」闭环，并沉淀为可复用团队资产。

鸿蒙「一多」工程实践（一次开发多端部署）与 Android 能力推演

背景 (Situation)：鸿蒙端需同时覆盖手机 / 双折叠 / 三折叠 / 平板多种形态，并应对横竖屏、折叠态等连续形变。早期各业务页面倾向按机型「各写一套」，UI 适配口径不一、重复代码多，KMP 车系页在横竖屏切换时还易出现实例重建、状态丢失。

职责 (Task)：作为多设备适配方案的设计与落地负责人，系统研究华为官方「一次开发、多端部署（一多）」理念与适配链路，在鸿蒙 + KMP 双仓沉淀统一的多设备适配规范与工具，并将其设计思想推演到 Android 侧。

举措 (Action)：

- **吃透官方一多链路**：依「三层工程架构（产品定制层 / 基础特性层 / 公共能力层）+ 自适应布局（拉伸 / 占比 / 缩放 / 隐藏 / 折行等）+ 响应式布局（断点 sm/md/lg + 媒体查询 + 栅格）」方法论，确立「连续形变用自适应、跨断点用响应式」的取舍原则。
- **鸿蒙侧体系化落地**：在双仓架构下搭建多设备 UI 适配规范，按断点统一栅格与布局策略，沉淀横竖屏 / 状态栏 / 挖孔区适配工具类收敛重复形变逻辑；针对 KMP 车系页横竖屏切换状态丢失，设计「进入态保持 + 中转退出」方案保障切换连续无跳变。
- **设计理念推演到 Android**：将「断点驱动响应式 + 能力分层下沉 + 安全区 / 形变统一处理」思想映射到 Android 折叠屏 / PAD / 横竖屏适配，在安卓侧落地对应的断点与适配能力，两端形成一致的多设备适配心智与口径。

结果 (Result)：形成可复用的鸿蒙多设备适配规范与工具集，重复适配代码与业务接入成本显著下降，横竖屏 / 折叠态切换状态丢失问题闭环；并将一多设计理念成功反哺 Android 侧落地对应能力，沉淀出跨平台一致的多形态适配方法论。

工作经历

懂车帝 · Android 高级开发工程师

2021.04 — 至今 · 北京

- 负责 **客户端性能与稳定性优化专项 (Android → 鸿蒙)**：Android 阶段参与 Native Crash / ANR / OOM / 卡顿等 Topic 排查并沉淀方法论，鸿蒙阶段作为专项 Owner，线上以 Slardar 为主、线下以 DevEco Profiler / SmartPerf 治理冷启动 / 卡顿丢帧 / 内存，形成每日性能优化日报与同口径复测机制
- 推进 **鸿蒙版从 0 到 1 建设与追平主端**：负责需求追赶节奏、跨团队（KMP/QA/UED/Server/华为）协作与人力协调，牵头小周会与周报机制，10 个月实现汽车垂类首个上架 App、85%+ 还原度
- 负责多设备适配（折叠屏 / 三折叠 / PAD），双折叠屏体验评分 4.79、三折叠屏 4.54，领先竞品
- 主导 HarmonyOS API17 → API20 大版本升级，制定可控升级节奏，升级期间主流程无重大线上事故
- 自研 **Android→鸿蒙 AI 辅助转换 (A2H) 方案**，借鉴规格化编码流水线思路（先文档后代码 / 实现与验证分离 / 失败自动回环）+ Android→鸿蒙映射知识库，显著提升 Android 到鸿蒙的页面转换效率

趣头条 · Android 开发工程师

2018.09 — 2021.04 · 上海

- 独立完成「爱走路」网赚 App 从 0 到 1 的整体搭建和上线
- 参与主 App 冷启动专项，拆分启动耗时点和启动线程治理
- 承担包体积压缩专项，将主包体积从 20M+ 压缩至约 11M

自如网 · Android 开发工程师

2015.09 — 2018.09 · 北京

- 参与自如客和管家 App 核心业务模块的功能开发、性能优化与架构演进

教育背景

延边大学 · 计算机科学与技术 · 全日制本科 · 2011 — 2015

下载 PDF 版本: [resume.pdf](#)